Phase 2, days 31 to 60: Design

EY: Talent Data Strategy

A0 Start here: foundations for Databricks

A0-L1 Spark DataFrames, the model and the core methods

Chain transformations and use their results, let Spark aggregate, and run an action only when you need the answer.

```python
from pyspark.sql import functions as F

df = spark.table("research_prod.research_silver.nat_respondents")
```

Transformation against action	Transformations (select, filter, withColumn, groupBy, join) describe a new DataFrame. Actions (show, count, collect, write) run the plan.
Why df.withColumn(...) on its own line does nothing	DataFrames are immutable. Use the result: df = df.withColumn(...).
Summarise per group	df.groupBy("region").agg(F.sum("w").alias("total"), F.count("*").alias("n")).
How to import Spark functions	from pyspark.sql import functions as F. Never import sum, max or min from it by name.

A0-L2 The SQL that pipelines use

Filter rows with WHERE and groups with HAVING, test NULL with IS NULL, and keep one row per key with ROW_NUMBER, not DISTINCT.

```sql
SELECT respondent_id, wave_id, final_weight
FROM (
  SELECT *,
         ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id ORDER BY
    delivery_version DESC) AS rn
  FROM bronze_deliveries
)
WHERE rn = 1
```

WHERE against HAVING	WHERE filters rows before grouping. HAVING filters groups after aggregation.
COUNT(*) against COUNT(column)	COUNT(*) counts rows. COUNT(column) counts non-NULL values. SUM and AVG skip NULLs.
Find rows with no match in a lookup	LEFT ANTI JOIN lookup ON key. A LEFT JOIN keeps all rows and fills NULLs.
Test for a missing value in SQL	IS NULL or IS NOT NULL. = NULL is never true.
Keep the latest row per key	ROW_NUMBER() OVER (PARTITION BY keys ORDER BY version DESC) AS rn, then WHERE rn = 1.

A0-L3 The lakehouse map, Databricks, Unity Catalog, Delta and medallion layers

Name every table in full, keep files in volumes, let Delta keep versions, and move data forward from bronze to silver to gold.

Unity Catalog name	catalog.schema.table, such as research_prod.research_silver.nat_respondents. Files live in volumes.
What makes a Delta table	Parquet data files plus _delta_log, where each write adds a numbered commit: versions 0, 1, 2.
Read an older version of a table	SELECT * FROM t VERSION AS OF 1. DESCRIBE HISTORY t lists the versions.
The three medallion layers	Bronze: as received, with audit columns. Silver: clean, one row per key. Gold: aggregates with a stated grain.

A2 Medallion architecture

A2-L1 Bronze: keep every delivery exactly as received

The strong version keeps every byte the supplier sent, labels every row with its origin, and reads the file once.

The four rules of the bronze contract	Append only. As received. Audit columns. Grain: respondent per wave per delivery file.
The four bronze audit columns	ingestion_id, source_file, ingestion_timestamp, delivery_version.
Why no inferSchema in bronze?	It guesses types and can destroy values, such as the leading zeros in 00123.
Rows written, without a second read	DeltaTable.forName(spark, t).history(1) and operationMetrics["numOutputRows"].

A2-L2 Silver: canonical names, checked types, one row per respondent per wave

The strong version fails early and clearly on a changed delivery, and turns bad values into a number you can check instead of a crash or a silent NULL.

```
F.col("final_weight").try_cast("double")
```

What does withColumnRenamed do with a missing column?	Nothing, silently. Check the source columns yourself and raise one error listing every missing one.
cast on Spark 4.0 (Runtime 17.3 LTS) with a malformed value	Raises CAST_INVALID_INPUT, because ANSI mode is on by default.
Safe cast for silver	F.col(c).try_cast(t), then count the new NULLs and check them against a threshold.
Silver dedup key	The full grain: respondent_id and wave_id.

A2-L3 Gold: state the grain, group by all of it, guard every ratio

The strong version cannot merge two trackers, reports a true zero for zero reach, and never divides by nothing.

```
COMMENT ON TABLE research_gold.nat_metrics IS 'Grain: tracker x wave x age_group x gender x region';
ALTER TABLE research_gold.nat_metrics SET TBLPROPERTIES ('grain' = 'tracker x wave x age_group x gender x region');
```

Where do you record a gold table's grain?	In the function docstring and on the table: COMMENT ON TABLE or a custom property such as 'grain'.
sum over no matching rows	Returns NULL, not 0. Wrap it: F.coalesce(F.sum(...), F.lit(0)).
Safe ratio in PySpark	F.when(base > 0, part / base). NULL when the base is 0.
Why is SET TBLPROPERTIES ('owner' = ...) wrong?	owner is reserved. Spark 4.0 raises UNSUPPORTED_FEATURE.SET_TABLE_PROPERTY. Use ALTER TABLE ... OWNER TO.

B18 Documentation and communication

B18-L1 Docstrings and READMEs that answer the next person's questions

Docstrings state grain, units, missing values and what None means. The README states purpose, quick start, inputs, config, tests and owners.

```python
def weighted_share(df: pd.DataFrame, value: str, weight: str) -> float | None:
    """Weighted mean of a 0/1 column. Grain: one row per respondent.

    Args:
        df: Respondent-level data.
        value: 0/1 column to average.
        weight: Non-negative weight column. Rows with a missing value or weight are dropped.

    Returns:
        The weighted share, or None when no rows remain or weights sum to 0.

    Raises:
        KeyError: If a named column is missing.
    """
```

Google-style docstring sections	One-line summary, then Args:, Returns:, Raises:.
What a data docstring must state	Grain, units, missing and zero handling, and what None means.
README sections	Purpose, quick start, inputs and outputs, configuration, tests, owners.
Keeping docs true	Update docstrings in the same commit as the code. Test the quick start where possible.

B18-L2 Model cards that state limits as clearly as scores

A model card states purpose, limits, data, honest metrics, subgroup results and monitoring, and is reviewed with every model change.

Model card sections	Details, intended use, out-of-scope uses, training data, evaluation, subgroups, limitations, monitoring.
How to report a metric on a card	With test period, prevalence and row count.
Why out-of-scope uses	They stop reuse for decisions the model was never validated for.
Keep cards honest	Template with required sections, reviewed in the model's pull request, checked in CI.

B18-L3 Decision logs, so the reasons outlive the meeting

One short, dated entry per decision, with options, consequences and status, linked from the code it explains, and superseded rather than edited.

```
## D-007: Trim survey weights at 5
Date: 2026-09-12. Status: accepted.
Context: Four respondents had weights above 12 and moved brand share by 4 points.
Options considered:
- No trimming
- Trim at 5, redistribute to keep the weighted total
- Rake with bounded weights
Decision: Trim at 5 with redistribution. Raking deferred until the new panel design.
Consequences: Design effect falls from 2.1 to 1.6. Small bias towards the sample
  profile. Review at wave 20.
```

Parts of a decision entry	ID and title, date and status, context, options considered, decision, consequences.
When a decision changes	Add a new entry. Mark the old one superseded and link to the new one.
Where and when to write	In the repository, in the same pull request as the change.
Make it findable	Link from the config value, the model card and the assumptions log.

C6 Semantic models and Direct Lake

C6-L1 Star schemas for semantic models

Facts with a stated grain, dimensions with unique keys, one-to-many single-direction relationships, and derived columns computed upstream.

Star schema parts	Fact tables (keys and measures, with a grain) and dimension tables (unique key, descriptive columns).
Relationship guidance	One-to-many, single direction. Minimise bi-directional relationships.
Calculated columns and Direct Lake	Not supported on SQL endpoints, preview on OneLake. Compute derived columns in gold.
Grain	Set by the fact table's dimension keys. State it in the table's docstring and comment.

C6-L2 DAX measures that state their filter context

Measures, not calculated columns. DIVIDE for every ratio, variables for clarity, and CALCULATE with the narrowest filter change.

```
Weighted aware % =
VAR aware_w = SUMX ( fact_awareness, fact_awareness[aware] * fact_awareness[weight] )
VAR base_w  = SUM ( fact_awareness[weight] )
RETURN DIVIDE ( aware_w, base_w )
```

Measure vs calculated column	Measures compute at query time in the visual's filter context. Calculated columns are stored at refresh.
DIVIDE in DAX	Returns BLANK (or an alternate result) when the denominator is zero or BLANK.
VAR ... RETURN	Evaluates once, names parts, improves performance, readability and debugging.

Share of total across brands	CALCULATE([measure], ALL(dim_brand)): remove only the brand filter.

C6-L3 Direct Lake, framing and the fallback you did not plan

Direct Lake stays fast when tables are Delta (not views), compact, within the SKU's guardrails, and fallback is made visible in development.

Direct Lake refresh	Framing: records Delta table versions in seconds. No data copy.
Direct Lake on OneLake vs on SQL	OneLake: never falls back. SQL: falls back to DirectQuery for views and SQL-based security.
Direct Lake behavior values	Automatic (default, silent fallback), DirectLakeOnly (error), DirectQueryOnly (always DirectQuery).
F64 per-table guardrails	5,000 Parquet files, 1.5 billion rows, 25 GB max memory.

A3 Delta Lake

A3-L1 Idempotent MERGE, and why you deduplicate the source first

The strong version is idempotent for any source: repeated keys, a retry or a full redelivery. The weak version works only while the supplier never repeats a row.

When is a pipeline idempotent?	When running it twice with the same input leaves the table as one run would.
Two conditions for an idempotent Delta MERGE	The ON condition covers the full grain, and the source has at most one row per key.
What happens when two source rows match one target row?	The MERGE fails with DELTA_MULTIPLE_SOURCE_ROW_MATCHING_TARGET_ROW_IN_MERGE.
Why not dropDuplicates() before a MERGE?	It keeps an arbitrary row per key. Use row_number() over a window ordered by version, newest first.

A3-L2 Schema enforcement is on by default. Evolve on purpose.

Schema changes become decisions a person reviews, not accidents a delivery causes.

Delta append with an extra column, no options	Fails with a schema mismatch. Enforcement is on by default.
MERGE with updateAll and an extra source column, no evolution	Succeeds and silently ignores the extra column. Only your own column check catches it.
Evolve one write on purpose	Append: option mergeSchema. MERGE: .withSchemaEvolution(). Planned: ALTER TABLE ADD COLUMN.
Why not set autoMerge.enabled for the session?	It turns evolution on for every MERGE in the job, including ones you did not intend.

A3-L3 Time travel, RESTORE, OPTIMIZE and VACUUM

History turns a data incident into a two-minute restore. Keep it long enough to be useful.

Read an old Delta version	VERSION AS OF n or TIMESTAMP AS OF ts in SQL; option versionAsOf in PySpark.
What does RESTORE do to history?	Adds a new version with the old content. Nothing is deleted.
Default retention windows	Deleted files 7 days (delta.deletedFileRetentionDuration). Log 30 days (delta.logRetentionDuration).
Why is VACUUM ... RETAIN 0 HOURS dangerous?	It deletes the files that time travel and RESTORE need, and it breaks readers still using them.
Databricks' recommendation for layout and maintenance on UC managed tables	Liquid clustering (CLUSTER BY or CLUSTER BY AUTO) with predictive optimization.

C9 Governance and security

C9-L1 Domains and endorsement, finding the right data

Domains help people find data. Endorsement tells them which copy to trust. Neither controls access.

What a domain does	Groups workspaces by business area for discovery and delegated settings. Does not control access.
Domain roles	Fabric admin creates domains. Domain admin edits one. Domain contributor, a workspace admin, assigns their workspaces.
Endorsement badges	Promoted (creators), Certified (named reviewers), Master data (named users, data items only). Dashboards get none.
Does endorsement block use of other copies?	No. It adds a badge and ranks the item first in some lists.

C9-L2 OneLake security, roles down to rows and columns

Readers are Viewers, defaults are removed, related row and column rules share a role, and every engine runs in a mode that enforces them.

OneLake security combines roles how?	Within a role: intersection of table, row and column rules. Across roles: union. Row filters OR together.
Whom does OneLake security restrict?	Viewers and Read-permission users. Admins, Members and Contributors already have data access.
SELECT * with a hidden column	Spark returns allowed columns. SQL analytics endpoint returns an error.
Engines enforcing OneLake RLS and CLS (GA)	Spark, lakehouse, Direct Lake on OneLake, SQL analytics endpoint in user's identity mode. Eventhouse: RLS only, preview.

C9-L3 Sensitivity labels, Purview and lineage

Run impact analysis before changes. Label sensitive items, attach protection policies, and let inheritance carry the label downstream.

Downstream label inheritance rules	Flows to downstream items. Never overwrites a manual label. Never downgrades.

Protection policy	Tied to a label. Named users and groups keep existing permissions. Everyone else is blocked.
Lineage view	Every workspace has one. All items in the workspace plus external sources one step upstream.
Impact analysis	Downstream items and workspaces a change might affect, with a way to notify their contacts.

A1 Code structure

A1-L1 Single-purpose functions and clear names

Each strong function passes the one-sentence test. The weak one needs "and" four times.

The five jobs a pipeline function does, one at a time	Read, transform, validate, write, log. An orchestrator calls them in order.
The one-sentence test	Describe the function without using "and". If you need "and", split it.
Function, class and constant naming	check_table_health, CheckResult, MAX_WEIGHT_CV.
What does a get_ or check_ prefix promise?	No side effects. The function reads and returns. It never writes or deletes.

A1-L2 Type hints and docstrings a reviewer can trust

The strong version tells mypy, the reviewer and the caller the same story. The weak version tells three different ones.

```python
def weighted_share(df: DataFrame, flag_col: str) -> float | None:
    """Weighted share of rows where flag_col is 1.

    Args:
        df: Respondents with final_weight.
        flag_col: 0/1 column to share.

    Returns:
        Share between 0 and 1, or None when the total weight is 0.

    Raises:
        ValueError: If flag_col is missing from df.
    """
```

Optional parameter that defaults to None	param: str | None = None. Never param: str = None.
A fixed set of string values	Literal["PASS", "WARN", "FAIL"] from typing.
Google docstring sections	One-line summary, then Args, Returns, Raises.
Why merge_keys: list[str] and not list?	mypy then rejects a string, which Python would split into characters.

A1-L3 Config over hardcoded values, and PEP 8 with ruff

The strong version runs unchanged in dev and prod. The contract file decides the table and the limits.

What belongs in configuration?	Anything that changes between environments, trackers or contract versions: names, maps, thresholds, rules.
When do you validate configuration?	Once, on load, before any data moves. Fail fast with the key's name.
Why not raw.get("catalog", "research_prod")?	A default hides a missing key and can point a run at production.
The two ruff commands for CI	ruff check . and ruff format --check .

A6 Data quality

A6-L1 Data contracts with severity levels and structured results

Results are data, severity decides the action, and one run reports every problem.

```python
CheckName = str
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]

@dataclass(frozen=True)
class CheckResult:
    check_name: CheckName
    status: Status
    severity: Severity
    expected: str
    actual: str
```

What does a contract check produce?	A structured record: check_name, status, severity, expected, actual. Never a sentence.
CRITICAL versus WARN	CRITICAL failures stop the pipeline by raising. WARN results are logged and the run continues.
Why evaluate every check before raising?	One run reports every problem, so the supplier fixes everything in one round trip.
Lakeflow Declarative Pipelines expectations	@dp.expect keeps and records, @dp.expect_or_drop drops, @dp.expect_or_fail stops the update.

A6-L2 Statistical checks that survive empty, single-row and zero data

The strong version returns a result for every wave and every subgroup, and says why when a statistic is undefined.

F.stddev on one row, and mean on an empty frame	Both NULL. stddev is the sample standard deviation.
Division by zero on Spark 4.0 with ANSI on	Raises DIVIDE_BY_ZERO, even for DOUBLE. Use F.try_divide or F.when(b > 0, a / b).
What should a check return when a statistic is undefined?	An explicit SKIP result with the reason. Never nothing, and never a crash.
Why is `if cv:` wrong?	A real CV of 0.0 is falsy. Test missing values with `is None`.

A6-L3 Reconcile counts and totals between layers

Silver and gold totals are computed the same way and compared with tolerance. A mismatch becomes a CRITICAL result with numbers, not a print.

Bronze figure that must match silver's row count for a wave	Distinct (respondent_id, wave_id) keys, not raw bronze rows, because bronze keeps every delivery.
Silver-to-gold reconciliation	In-scope silver weight total = sum of gold weighted_base; in-scope silver rows = sum of unweighted_n.
How to compare floating-point totals	math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6). Never ==.
Severity of a failed reconciliation	CRITICAL. Every figure for the wave is suspect until it passes.

D4 Data errors

D4-L1 Row counts across layers

Write the expected relationship between layers as rules, compute counts in one pass on identical filters, and halt when a CRITICAL rule fails.

Row-count rules: silver vs bronze	Silver rows = distinct respondent keys in bronze for the tracker and wave.
Rows after a MERGE	Rows before + numTargetRowsInserted (minus any deletes). Updates add nothing.
Counting efficiently	One agg pass: F.count('*') and F.count_distinct(key). Not repeated count() calls.
The get_unprocessed_waves bug	Bronze filtered on tracker, silver not. Filter both sides on the same keys.

D4-L2 Distribution drift between waves

Compare shares over the union of categories, flag new and missing values, summarise with PSI, and guard every total.

```
PSI = sum over categories of (actual - expected) x ln(actual / expected)
```

Three drift checks	Category shares vs prior wave, new and missing categories, and a PSI summary.
PSI formula	Sum of (actual - expected) x ln(actual / expected) over shares, with zero shares floored.
PSI thresholds 0.1 and 0.25	A rule of thumb, not a standard. Calibrate on your own wave history.
Percentage points vs relative change	30% to 45% is +15 points and +50% relative. Say which a tolerance uses.

D4-L3 Weight totals and survey checks

Check nulls, range, sum and efficiency per wave and subgroup. Mind Spark's nulls: sample stddev, empty aggregates and sums over no matches.

```
deff  = n x sum(w^2) / (sum w)^2 = 1 + CV^2
n_eff = (sum w)^2 / sum(w^2) = n / deff
```

Kish design effect and effective sample	deff = n x sum(w^2) / (sum w)^2 = 1 + CV^2. n_eff = (sum w)^2 / sum(w^2).

F.stddev in Spark	Sample standard deviation, null for one row. Use F.stddev_pop for 1 + CV^2.
sum(when(cond, 1)) with no matches	Null, not 0. Wrap in coalesce(..., lit(0)) or add otherwise(0).
Severity for a null weight	CRITICAL: halt before gold.

A9 Testing

A9-L1 pytest basics, and a SparkSession fixture shared by every test

One session-scoped fixture, and tests with known answers that fail for the bugs you care about.

```python
def test_weighted_share_uses_weights() -> None:
    assert weighted_share([1, 0], [3.0, 1.0]) == 0.75
```

```python
@pytest.fixture(scope="session")
def spark() -> Iterator[SparkSession]:
    session = SparkSession.builder.master("local[2]").getOrCreate()
    yield session
    session.stop()
```

```python
@pytest.mark.parametrize(("flags", "weights", "expected"), [([1], [2.0], 1.0), ([0, 0], [1.0, 1.0], 0.0)])
def test_share(flags, weights, expected): ...
```

What does pytest run?	Functions named test_* in files named test_*.py. Plain assert statements.
SparkSession fixture scope	scope="session": start Spark once per test run.
yield in a fixture	Hands the value to tests. Code after yield is the teardown.
What makes a test worth having?	It fails for the bug you fear: known answers that separate right from wrong, plus edge cases.

A9-L2 A fast local SparkSession and readable test data

A tuned local session makes tests fast enough to run on every change. Builders with explicit schemas make them short and exact.

```python
builder = (
    SparkSession.builder.master("local[2]")
    .config("spark.sql.shuffle.partitions", "2")
    .config("spark.ui.enabled", "false")
    .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
    .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
    .config("spark.sql.sources.default", "delta")
)
spark = configure_spark_with_delta_pip(builder).getOrCreate()
```

```python
def respondent(**overrides: object) -> dict[str, object]:
    row = {"respondent_id": "R1", "wave_id": "W1", "final_weight": 1.0, "in_scope": True}
    row.update(overrides)
    return row
```

Local test session settings	master("local[2]"), shuffle partitions 2, UI off, Delta configured with configure_spark_with_delta_pip.
Default spark.sql.shuffle.partitions	200: far too many for test data.

Why explicit schemas in test data?	Inference fails on all-None columns and can pick different types from production.
Test data builder	A function with valid defaults. Tests override only the fields they are about, and typos raise.

A9-L3 Compare DataFrames properly, and test that MERGE is idempotent

One helper compares DataFrames the right way, and one small test proves a MERGE survives a retry and a repeated key.

```
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual

assertDataFrameEqual(actual, expected)
```

Compare two DataFrames in a test	pyspark.testing.assertDataFrameEqual(actual, expected). Ignores order, tolerates float rounding, checks schema.
When should row order matter?	Only for ranked output. Pass checkRowOrder=True.
Idempotence test for a MERGE	Load, snapshot with collect(), load again, assertDataFrameEqual(table, snapshot).
Why collect() for the snapshot?	A DataFrame is lazy and would re-read the table after the second run.